Smart Pay — Complete Technical Walkthrough

Every flow traced through the actual code

---

1. APPLICATION STARTUP

When you run `uvicorn main:app --reload`, here's what happens:

File: `main.py`

```
from app.controllers import auth_controller, invoice_controller,
prediction_controller, csv_controller
```

This imports all 4 controllers. During import, each controller creates its service instances.

File: `app/models/database.py` (bottom of file)

```python
engine = create_engine(f"sqlite:///{DATABASE_PATH}", ...)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
init_db()  # ← runs on import, creates tables if they don't exist
```

SQLAlchemy connects to `smartpay.db` and creates the 3 tables (users, invoices, invoice_features) if they don't already exist.

File: `app/controllers/invoice_controller.py` (line ~15)

```python
csv_service = CSVService()  # ← triggers CSV loading on startup
```

File: `app/services/csv_service.py` → `__init__()` → `_load_all_csv_data()`

```python
def _load_all_csv_data(self):
    # 1. Scans app/data/ folder for CSV files
    csv_files = list(DATA_DIR.glob("*.csv"))

    # 2. For each CSV, calls _load_csv_file()
    for f in csv_files:
        self._load_csv_file(f)
```

File: `app/services/csv_service.py` → `_load_csv_file()`

```python
def _load_csv_file(self, csv_path):
    df = pd.read_csv(csv_path)
    cols_lower = [c.lower().strip() for c in df.columns]

    # DETECTION: Checks which format the CSV is
    if 'dayslate' in cols_lower or 'daystosettle' in cols_lower:
        self._load_ibm_dataset(df, csv_path.name)    # ← IBM format
    elif 'late_payment' in cols_lower:
        self._load_dataset_csv(df, csv_path.name)     # ← Custom format
    else:
        self._load_customer_history_csv(df, csv_path.name)  # ← Generic
```

The system auto-detects the IBM dataset by looking for columns like `DaysLate`, `DaysToSettle`, `countryCode`.

File: `app/services/csv_service.py` → `_load_ibm_dataset()`

This is where the IBM data gets cleaned and processed:

```python
```

```python
def _load_ibm_dataset(self, df, source):
    # STEP 1: Map column names (case-insensitive)
    # 'customerID' → 'customer_id'
    # 'invoiceAmount' → 'amount'
    # 'DaysLate' → 'days_late'
    # 'Disputed' → 'disputed'

    # STEP 2: Map anonymised IDs to company names
    if 'customer_name' not in df.columns and 'customer_id' in df.columns:
        df['customer_name'] = df['customer_id'].apply(self._id_to_company_name)
    # "3993-QUNVJ" → "Apex Trading"
    # Same ID always maps to the same name (stored in _id_map dict)

    # STEP 3: Create binary label from DaysLate
    if 'late_payment' not in df.columns and 'days_late' in df.columns:
        df['late_payment'] = (df['days_late'] > 0).astype(int)
    # DaysLate = 15 → late_payment = 1 (late)
    # DaysLate = 0  → late_payment = 0 (on time)

    # STEP 4: Build customer profiles
    for _, row in df.iterrows():
        name = str(row.get('customer_name', '')).strip()

        # Initialise customer if first time seeing them
        if name not in self.customer_history:
            self.customer_history[name] = {
                'name': name,
                'total_invoices': 0,
                'paid_on_time': 0,
                'paid_late': 0,
                'not_paid': 0,
                'total_amount': 0.0,
                'avg_payment_delay': 0.0,
                'risk_factors': []
            }

        cust = self.customer_history[name]
        cust['total_invoices'] += 1
        cust['total_amount'] += amount

        # Count late vs on-time
        if late == 1:
            cust['paid_late'] += 1
            # Update running average delay
        else:
            cust['paid_on_time'] += 1
```

```
        # Extract risk factors
        if disputed:
            cust['risk_factors'].append('Disputed invoice')
```

After startup, you have:

- `self.customer_history` = dictionary with ~100 customers, each having their payment stats
- `self.dataset_records` = list of all 2,466 raw records
- Terminal shows: "Loaded 100 customers, 2466 records"

---

2. USER REGISTRATION

Flow: User fills form → POST /register

File: `app/controllers/auth_controller.py`

python

```python
@router.post("/register")
async def register(request: Request, email: str = Form(...),
                   password: str = Form(...), ...):

    # STEP 1: Sanitise inputs (prevent XSS)
    email = sanitize(email)  # strips <script> tags, quotes, brackets

    # STEP 2: Validate email format
    # Uses RFC 5322 regex: r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

    # STEP 3: Validate password strength
    # Minimum 8 chars, at least 1 uppercase, 1 lowercase, 1 number

    # STEP 4: Check if email already exists in database
    existing = db.query(User).filter_by(email=email).first()

    # STEP 5: Hash password with bcrypt (12 rounds)
    import bcrypt as _bcrypt
    hashed = _bcrypt.hashpw(password.encode('utf-8'), _bcrypt.gensalt(12))
    # "MyPass123" → "$2b$12$xK9z..." (60-char hash, different every time due to salt

    # STEP 6: Save to database
    user = User(email=email, password=hashed.decode('utf-8'))
    db.add(user)
    db.commit()

    # STEP 7: Redirect to login
    return RedirectResponse("/login", status_code=303)
```

File: `app/models/database.py` — User table

```python
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    email = Column(String(120), unique=True, nullable=False)
    password = Column(String(255), nullable=False)  # bcrypt hash stored here
    created_at = Column(DateTime, default=datetime.utcnow)
```

---

3. USER LOGIN

Flow: User submits email/password → POST /login

File: `app/controllers/auth_controller.py`

```python
@router.post("/login")
async def login(request: Request, email: str = Form(...), password: str = Form(...)):

    # STEP 1: Check brute force lockout
    ip = request.client.host
    # login_attempts is a dict: {"192.168.1.1": [timestamp1, timestamp2, ...]}
    recent = [t for t in login_attempts[ip] if now - t < 300]  # last 5 minutes
    if len(recent) >= 5:
        return "Account locked. Try again in 5 minutes."

    # STEP 2: Find user by email
    user = db.query(User).filter_by(email=sanitize(email)).first()

    # STEP 3: Verify password with bcrypt
    import bcrypt as _bcrypt
    if _bcrypt.checkpw(password.encode('utf-8'), user.password.encode('utf-8')):
        # Correct password!

        # STEP 4: Write session
        request.session["user_id"] = user.id
        request.session["email"] = user.email
        # This creates a signed cookie via Starlette SessionMiddleware

        # STEP 5: Clear failed attempts
        login_attempts[ip] = []

        return RedirectResponse("/dashboard", status_code=303)
    else:
        # Wrong password — record the failed attempt
        login_attempts[ip].append(time.time())
```

Why bcrypt directly and not passlib? passlib has a bug on Python 3.13 where `bcrypt.verify()` silently returns False even for correct passwords. Using `bcrypt.checkpw()` directly works correctly.

---

4. INVOICE UPLOAD & OCR EXTRACTION

Flow: User uploads PDF → POST /upload → OCR extracts data → saved to DB

File: `app/controllers/invoice_controller.py` → `upload_invoice()`

```python
```

```python
@router.post("/upload")
async def upload_invoice(request: Request, file: UploadFile = File(...)):

    # STEP 1: Validate file type
    allowed = {'.pdf', '.jpg', '.jpeg', '.png', '.bmp', '.tiff', '.tif'}
    ext = Path(file.filename).suffix.lower()
    if ext not in allowed: return error

    # STEP 2: Sanitise filename (prevent path traversal)
    safe_name = re.sub(r'[^a-zA-Z0-9._\- ]', '', file.filename)
    # "../../etc/passwd" → "etcpasswd"
    # "invoice<script>.pdf" → "invoicescript.pdf"

    # STEP 3: Check file size (max 10MB)
    if len(content) > 10 * 1024 * 1024: return error

    # STEP 4: Save file to disk
    file_path = UPLOAD_DIR / safe_name
    with open(file_path, "wb") as f:
        f.write(content)

    # STEP 5: Extract data using OCR
    extracted = extract_data(file_path)

    # STEP 6: Determine initial status
    due = _parse_date(extracted.get('due_date', 'N/A'))
    initial_status = 'overdue' if (due and due < date.today()) else 'pending'

    # STEP 7: Save to database
    invoice = Invoice(
        filename=safe_name,
        invoice_number=sanitize(extracted['invoice_number']),
        customer_name=sanitize(extracted['customer_name']),
        amount=extracted['amount'],
        currency=extracted['currency'],
        ...
        status=initial_status,
    )
    db.add(invoice)
    db.commit()
```

File: `app/controllers/invoice_controller.py` → `extract_text()`

```python
```

```python
def extract_text(file_path):
    suffix = file_path.suffix.lower()

    if suffix == '.pdf':
        # Uses PyMuPDF (fitz) library
        import fitz
        doc = fitz.open(file_path)
        for page in doc:
            # Structured extraction — reads text blocks with position info
            blocks = page.get_text("dict").get("blocks", [])
            # Sorts blocks top-to-bottom, left-to-right
            # This preserves the layout better than plain text extraction

    elif suffix in ['.png', '.jpg', '.jpeg']:
        # Uses Tesseract OCR
        import pytesseract
        from PIL import Image, ImageEnhance, ImageFilter
        img = Image.open(file_path)

        # Preprocessing for better OCR accuracy:
        img = img.convert('L')                    # Convert to grayscale
        img = ImageEnhance.Contrast(img).enhance(2.0)   # Increase contrast
        img = ImageEnhance.Sharpness(img).enhance(2.0)  # Sharpen
        # Scale up small images (OCR works better on larger images)

        text = pytesseract.image_to_string(img)
```

File: `app/controllers/invoice_controller.py` → `extract_data()`

Calls individual field extractors:

python

```python
def extract_data(file_path):
    text = extract_text(file_path)

    data['invoice_number'] = _find_invoice_number(text)
    data['customer_email'] = _find_email(text)
    data['amount'], data['currency'] = _find_amount_and_currency(text)
    data['date'], data['due_date'] = _find_dates(text)
    data['customer_name'] = _find_customer_name(text)
    data['company_name'] = _find_company_name(text)
    data['vat_id'] = _find_vat_id(text)
```

Key extraction functions:

python

```python
def _find_invoice_number(text):
    # Tries these patterns in order:
    # "Invoice # INV-C-2026-13534327" → captures "INV-C-2026-13534327"
    # "Invoice Number: 12345" → captures "12345"
    # "Bill No: BILL-999" → captures "BILL-999"

def _find_amount_and_currency(text):
    # Detects currency: € → EUR, $ → USD, £ → GBP
    # Finds amount near "Total", "Amount Due", "Invoice Amount"

    # CRITICAL: _parse_amount() handles EU vs US format:
    # "9,84"     → 9.84  (European: comma is decimal)
    # "1,234.56" → 1234.56 (US: comma is thousands)
    # "1.234,56" → 1234.56 (European: dot is thousands, comma is decimal)

def _find_dates(text):
    # Tries LABELED dates first:
    # "Invoice Date: Jan 18, 2026" → captures "Jan 18, 2026"
    # "Due Date: Feb 18, 2026" → captures "Feb 18, 2026"
    # "Next Billing Date: Feb 18, 2026" → captures as due date

    # Falls back to finding ANY dates in the text

def _find_customer_name(text):
    # Looks for "BILLED TO" section, takes the first line after it
    # Falls back to deriving name from email:
    # "hadi24107@gmail.com" → "Hadi"

def _find_company_name(text):
    # Scans first 10 lines for company suffixes:
    # Ltd, LLC, Inc, Corp, GmbH, SA, SL, BV, Company
    # "Freepik Company, SL" → match!
```

---

5. ML MODEL TRAINING

Flow: User clicks "Train" at /ml/train

File: `app/controllers/prediction_controller.py` → `train_model()`

python

```python
@router.get("/ml/train")
def train_model(request: Request):
    count, message = ml_service.train_from_db(db, csv_path)
```

File: `app/services/ml_service.py` → `train_from_db()`

```python
```

```python
def train_from_db(self, db, csv_path):
    # STEP 1: Read CSV files from app/data/
    for csv_file in DATA_DIR.glob("*.csv"):
        df = pd.read_csv(csv_file)

        # STEP 2: Detect IBM format
        if 'dayslate' in cols:
            # For each row, build a feature vector:
            feature_vec = [
                amount / 100000,        # Feature 1: normalised amount
                0.5,                     # Feature 2: default reliability
                days_late / 60,          # Feature 3: normalised days late
                days_settle / 90,        # Feature 4: normalised settlement time
                0.0,                     # Feature 5: late ratio (placeholder)
                disputed,                # Feature 6: disputed flag (0 or 1)
                0.0                      # Feature 7: risk factor count
            ]
            label = 1 if days_late > 0 else 0  # 1 = late, 0 = on time

    # STEP 3: Split data 80/20
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
    # random_state=42 means same split every time = reproducible results

    # STEP 4: Train Random Forest
    self.model = RandomForestClassifier(
        n_estimators=50,          # 50 decision trees vote together
        max_depth=6,              # Each tree max 6 levels deep (prevents overfittin
        random_state=42,          # Reproducible
        class_weight='balanced'   # Equal weight to late and on-time classes
    )
    self.model.fit(X_train, y_train)

    # STEP 5: Test accuracy
    accuracy = accuracy_score(y_test, self.model.predict(X_test))
    # e.g. "Trained with 2466 samples. Accuracy: 72.3%"

    # STEP 6: Save model to disk
    joblib.dump(self.model, self.model_path)
    # Saves to app/models/payment_predictor.pkl
    # Next time server starts, it loads this file instead of retraining
```

Why Random Forest instead of a single Decision Tree?

- A single Decision Tree memorises the training data (overfitting)

- Random Forest trains 50 trees, each on a different random subset
- When predicting, all 50 trees vote → majority wins
- This averages out individual tree mistakes → more robust predictions

---

6. PAYMENT PREDICTION

Flow: User clicks "Predict" on an invoice → GET /ml/predict/{id}

File: `app/controllers/prediction_controller.py` → `predict_payment()`

```python
@router.get("/ml/predict/{invoice_id}")
def predict_payment(request, invoice_id):
    invoice = db.query(Invoice).filter(Invoice.id == invoice_id).first()

    # STEP 1: Get base ML prediction
    proba, label = ml_service.predict_for_invoice(db, invoice_id)
```

File: `app/services/ml_service.py` → `predict_for_invoice()`

```python
```

```python
def predict_for_invoice(self, db, invoice_id):
    invoice = db.query(Invoice).filter(Invoice.id == invoice_id).first()
    customer_match = self.csv_service.find_customer_match(invoice.customer_name)

    if customer_match:
        # ═══ KNOWN CUSTOMER ═══
        # Build feature vector from REAL history
        risk = self.csv_service.calculate_payment_risk(customer_match, amount)

        feature_vector = [
            amount / 100000,                                # normalised amount
            risk.get('payment_reliability', 0.5),           # actual reliability from
            customer_match.get('avg_payment_delay') / 60, # actual avg delay
            min(total_invoices, 100) / 100,                 # history depth
            paid_late / total_invoices,                      # actual late ratio
            1 if 'Disputed invoice' in risk_factors else 0, # disputed flag
            len(risk_factors) / 10                           # risk factor count
        ]

        # Run through Random Forest
        proba = self.model.predict_proba([feature_vector])[0][1]
        # predict_proba returns [prob_on_time, prob_late]
        # [0][1] = probability of late payment

        # BLEND: 60% model + 40% historical risk
        blended = 0.6 * proba + 0.4 * historical_risk
        return blended, 1 if blended > 0.5 else 0

    else:
        # ═══ UNKNOWN CUSTOMER ═══
        base_risk = 0.5  # default
        if amount > 50000: base_risk = 0.6
        elif amount > 20000: base_risk = 0.55
        elif amount < 5000: base_risk = 0.4

        feature_vector = [amount/100000, 0.5, 0.0, 0.0, 0.0, 0.0, 0.0]
        proba = self.model.predict_proba([feature_vector])[0][1]

        # BLEND: 40% model + 60% heuristic (less trust in model for unknowns)
        blended = 0.4 * proba + 0.6 * base_risk
        return blended, 1 if blended > 0.5 else 0
```

Back in `prediction_controller.py` → adjustments after ML:

```python
```

```python
    # STEP 2: Overdue boost
    overdue_days = _days_overdue(invoice.due_date)
    if overdue_days > 0:
        proba = min(1.0, proba + min(0.4, overdue_days * 0.01))
        # 10 days overdue → +10% risk
        # 30 days overdue → +30% risk
        # 40+ days overdue → capped at +40%

    # STEP 3: Unknown customer adjustment
    customer_match = csv_service.find_customer_match(invoice.customer_name or "")
    if not customer_match:
        proba = max(proba, 0.45)  # Floor: unknown ≠ safe

        due = _parse_date(invoice.due_date)
        if due:
            days_until = (due - date.today()).days
            if days_until <= 0: proba = max(proba, 0.65)   # Overdue
            elif days_until <= 3: proba = max(proba, 0.55)  # Due very soon
            elif days_until <= 7: proba = max(proba, 0.50)  # Due this week

        # Amount boost for unknowns
        if amt > 10000: proba += 0.15   # Big invoice from stranger = risky
        elif amt > 5000: proba += 0.10
        elif amt > 1000: proba += 0.05

        label = 1 if proba > 0.5 else 0

    # STEP 4: Save prediction to database
    features.predicted_probability = proba  # e.g. 0.55
    features.predicted_label = label        # 1 = high risk, 0 = low risk
    db.commit()
```

---

7. CUSTOMER RISK SCORING

File: `app/services/csv_service.py` → `calculate_payment_reliability()`

python

```python
def calculate_payment_reliability(self, data):
    # Base: what % of invoices were paid on time?
    reliability = paid_on_time / total_invoices
    # e.g. 15 on time out of 20 total = 0.75

    # Penalty for unpaid invoices
    if not_paid > 0:
        reliability *= max(0.5, 1 - (not_paid / total * 0.5))
    # 3 unpaid out of 20 = reliability × 0.925

    # Penalty for high average delay
    if avg_delay > 30: reliability *= 0.7    # Heavy penalty
    elif avg_delay > 15: reliability *= 0.85  # Moderate penalty
    elif avg_delay > 7: reliability *= 0.95   # Light penalty

    return max(0.1, min(1.0, reliability))  # Always between 10% and 100%
```

File: `app/services/csv_service.py` → `calculate_payment_risk()`

python

```python
def calculate_payment_risk(self, data, invoice_amount):
    reliability = self.calculate_payment_reliability(data)
    risk_score = 1 - reliability  # Flip: high reliability = low risk

    # Adjust for unusually large invoices
    avg_amount = total_amount / total_invoices
    if invoice_amount > avg_amount * 2:
        risk_score *= 1.2  # 20% boost for invoices 2x normal size

    # Adjust for risk factors
    risk_score *= (1 + len(risk_factors) * 0.1)
    # Each risk factor (Disputed, Negative cash flow, etc.) adds 10%

    # Classify
    if risk_score > 0.7: level = "High"
    elif risk_score > 0.4: level = "Medium"
    else: level = "Low"
```

File: `app/services/csv_service.py` → `find_customer_match()`

python

```python
def find_customer_match(self, name):
    # STAGE 1: Exact match
    # "Apex Trading" == "Apex Trading" ✓

    # STAGE 2: Case-insensitive partial match
    # "apex" found in "Apex Trading" ✓

    # STAGE 3: Cleaned match (remove special chars)
    # "OBrien" matches "O'Brien & Sons" ✓
    # Both become "obriensons" after cleaning

    # If no match found → returns None (unknown customer)
```

---

8. STATUS WORKFLOW

File: `app/controllers/invoice_controller.py`

Auto-detection on page load:

python

```python
def _check_overdue(invoice):
    if invoice.status in ('paid', 'disputed'):
        return invoice.status  # Don't override manual statuses

    due = _parse_date(invoice.due_date)
    if due and due < date.today():
        return 'overdue'  # Past due → automatically overdue
    return 'pending'      # Not past due → pending
```

Batch update on dashboard/invoice list:

python

```python
def _auto_update_statuses(db):
    pending = db.query(Invoice).filter(Invoice.status.in_(['pending', None])).all()
    for inv in pending:
        new_status = _check_overdue(inv)
        if new_status != inv.status:
            inv.status = new_status  # Flip pending → overdue if past due
    db.commit()
```

Manual status change:

python

```python
@router.post("/update-status/{invoice_id}")
async def update_status(request, invoice_id, status: str = Form(...)):
    valid = {'pending', 'paid', 'overdue', 'disputed'}
    invoice.status = status
    db.commit()
    # User clicks "Mark Paid" → status changes to 'paid'
    # User clicks "Dispute" → status changes to 'disputed'
    # User clicks "Reopen" → status changes back to 'pending'
```

---

9. COMPLETE PREDICTION EXAMPLE

Invoice: Freepik, €9.84, due 2026-03-19, unknown customer

1. `ml_service.predict_for_invoice()` runs:
   - Customer "hadi24107" not found in customer_history → unknown
   - amount = 9.84, base_risk = 0.4 (small amount)
   - feature_vector = [9.84/100000, 0.5, 0.0, 0.0, 0.0, 0.0, 0.0]
   - model.predict_proba → ~0.05 (very low, model thinks neutral)
   - blended = 0.4 × 0.05 + 0.6 × 0.4 = 0.26 (26%)
   - Returns: proba=0.26, label=0
2. Back in `predict_payment()`:
   - overdue_days = 0 (due date is tomorrow, not past)
   - Unknown customer check: proba = max(0.26, 0.45) → **0.45**
   - Due date check: days_until = 1 day → proba = max(0.45, 0.55) → **0.55**
   - Amount check: €9.84 < €1000 → no boost
   - label = 1 (0.55 > 0.5 = HIGH RISK)
   - **Final: 55% HIGH RISK**

If same invoice was from "Apex Trading" (known customer, 22 invoices, 0 on time, 22 late):

- reliability = 0/22 = 0.0, with delay penalty → 0.1 (minimum)
- risk_score = 1 - 0.1 = 0.9
- model prediction with real features → ~0.85
- blended = 0.6 × 0.85 + 0.4 × 0.9 = 0.87
- **Final: 87% HIGH RISK**

If same invoice was from "Summit Corp" (known, 16 invoices, 16 on time, 0 late):

- reliability = 16/16 = 1.0

- risk_score = 1 - 1.0 = 0.0
- model prediction → ~0.08
- blended = 0.6 × 0.08 + 0.4 × 0.0 = 0.048
- **Final: 5% LOW RISK**

---

10. FILE MAP — WHERE EVERYTHING LIVES

```
smartpay/
│
├── main.py                    ← App entry point, mounts all routers
│
├── app/
│   ├── controllers/
│   │   ├── auth_controller.py       ← Register, login, logout, brute force
│   │   ├── invoice_controller.py    ← Dashboard, upload, CRUD, OCR extraction,
│   │   │                              status workflow, overdue detection
│   │   ├── csv_controller.py        ← CSV upload, customer list, customer
insights
│   │   └── prediction_controller.py ← ML train, predict, analytics, explain
│   │
│   ├── services/
│   │   ├── csv_service.py     ← Loads IBM data, builds customer profiles,
│   │   │                        reliability scoring, risk assessment, fuzzy
matching
│   │   ├── ml_service.py      ← Random Forest training, prediction, model
persistence
│   │   ├── paths.py           ← Centralised file paths (DATA_DIR, UPLOAD_DIR,
etc.)
│   │   └── state.py           ← Singleton csv_service instance
│   │
│   ├── models/
│   │   ├── database.py        ← SQLAlchemy models (User, Invoice,
InvoiceFeatures),
│   │   │                        DB connection, table creation
│   │   └── payment_predictor.pkl  ← Saved Random Forest model (binary)
│   │
│   ├── templates/             ← 20+ Jinja2 HTML templates
│   │   ├── dashboard.html     ← Dual-mode: guest landing / logged-in dashboard
│   │   ├── view_data.html     ← All invoices with status filters
│   │   ├── view_invoice.html  ← Single invoice with status buttons + prediction
│   │   ├── ml_analytics.html  ← Payment insights (business-friendly)
│   │   └── ...
│   │
```

```
|   ├── data/
|   |   └── WA_Fn-UseC_-Accounts-Receivable.csv  ← IBM dataset (2,466 records)
|   |
|   └── tests/                 ← 109 automated tests
|       ├── test_auth.py       ← 24 tests: email, password, XSS, brute force,
bcrypt
|       ├── test_csv_service.py ← 24 tests: reliability, risk, matching,
safe_float
|       ├── test_invoice_extraction.py ← 26 tests: regex patterns, amounts, dates
|       ├── test_ml_service.py  ← 16 tests: features, training, accuracy, blending
|       └── test_utils_security.py ← 19 tests: filenames, file types, sessions
|
├── smartpay.db               ← SQLite database file
├── Dockerfile                ← Docker containerisation
├── requirements.txt          ← Python dependencies
└── pytest.ini                ← Test configuration
```